

PROJECT REPORT

Library Management System

A menu-driven, console-based Python application for book issue, return, waiting list and late-fine management

Submitted by	Aditi Rastogi
Registration Number	26BAI10100
Course / Program	CSE1021
Institution	VIT Bhopal
Submitted to	Siddharth Singh Chouhan
Date	30 September 2026
GitHub Repository	https://github.com/aditirastogi-code/Library-management-system.git

Table of Contents

Table of Contents	2
2. Introduction	3
3. Problem Statement	3
4. Functional Requirements	3
5. Non-functional Requirements	4
6. System Architecture	4
7. Design Diagrams	5
8. Design Decisions & Rationale	9
9. Implementation Details	10
10. Screenshots / Results	11
11. Testing Approach	13
12. Challenges Faced	14
13. Limitations & Future Enhancements	15
14. Conclusion	15

2. Introduction

Libraries, even small ones, depend on a few repetitive tasks: recording which books they own, lending them to members, taking them back, and charging a fine when they come back late. Done by hand in a register, these tasks are slow and error-prone. Copies get miscounted, borrowers are forgotten, and the fine is worked out inconsistently.

The **Library Management System** is a Python program that automates these tasks through a simple text menu. A librarian can add books and inspect the current state of the collection, and students can issue books, return them, and queue for a book that is currently unavailable. Librarian actions are protected by an authorisation number, and the fine for a late return is computed automatically.

The project uses only the Python standard library and is split into three modules: an entry point (`Command.py`), a librarian module (`Librarian.py`) and a student module (`Student_access.py`). It is intended as a compact, readable demonstration of functions, modules, lists, dictionaries, loops and input handling.

3. Problem Statement

A small library needs a lightweight way to manage its books and borrowers without a large database or web infrastructure. The system must let authorised staff add books and view their status, let students borrow and return books using their registration number, keep track of how many copies remain, handle the case where every copy is already issued, and apply a late-return fine consistently.

Objectives

- Keep a record of every book with its ID, author, category and number of available copies.
- Restrict book entry and status viewing to the librarian through an authorisation number with a limited number of attempts.
- Allow students to issue and return books, updating the available count automatically.
- Maintain a waiting list, in order, for books that have no free copies.
- Calculate the late fine automatically according to a fixed rule.
- Use only the Python standard library so the program runs anywhere Python 3.8+ is installed.

4. Functional Requirements

ID	Requirement	Implemented in
FR1	The system shall display a menu with five commands (add, status, issue, return, exit) and repeat until the user exits.	<code>Command.py</code>
FR2	The system shall ask for the authorisation number before add-book and check-status, allowing four entries in total before denying access.	<code>Librarian.authorization()</code>
FR3	The librarian shall be able to add a book by entering its name, ID, author, category and number of copies.	<code>Librarian.enter_the_name_of_books()</code>

FR4	The librarian shall be able to view every book with its details, available copies, current borrowers and waiting list.	Librarian.check_status()
ID	Requirement	Implemented in
FR5	A student shall be able to issue a book by name and registration number if at least one copy is available; the available count shall decrease by one.	Student_acess.issue_a_book()
FR6	If no copy is available, the system shall offer the student a place on the waiting list and report their position.	Student_acess.issue_a_book()
FR7	The system shall report when the requested book does not exist in the library.	Student_acess.issue_a_book()
FR8	A student shall be able to return a book; the system shall confirm that this student borrowed it and then increase the available count by one.	Student_acess.return_a_book()
FR9	If a book was kept for more than 7 days, the system shall calculate and display a fine.	Student_acess.return_a_book()
FR10	Book names and registration numbers shall be matched without regard to letter case.	Both modules (lower())
FR11	Unknown menu commands shall produce an "Invalid command" message.	Command.py

5. Non-functional Requirements

Category	Requirement
Usability	Plain text menu with numbered options and clear prompts; no training needed.
Portability	Runs on any operating system with Python 3.8 or later; no third-party packages.
Performance	Book lookup is a linear scan of an in-memory list, which is more than adequate for the small collections this system targets.
Maintainability	Logic is separated into librarian and student modules with one function per operation, so each feature can be changed independently.
Security	Librarian operations are guarded by an authorisation number with a limited number of attempts. (The number is hard-coded; see Limitations.)
Reliability	Return is validated against the borrower list so copy counts cannot be inflated by a student who never borrowed the book.
Data persistence	Data is held in memory for the duration of a session only and is lost on exit.

6. System Architecture

The system follows a simple layered, modular structure. `Command.py` is the presentation and control layer: it prints the menu, reads the command and dispatches to the right function.

`Librarian.py` and `Student_acess.py` form the business-logic layer. The data layer is a single

Python list, `list_of_books`, created in the entry point and passed by reference to every function. Each book is a dictionary in that list.

Module	Role	Functions
Command.py	Entry point, menu loop, command dispatch, owns <code>list_of_books</code>	(script-level loop)
Module	Role	Functions
Librarian.py	Staff operations and access control	<code>authorization()</code> , <code>enter_the_name_of_books()</code> , <code>check_status()</code>
Student_acess.py	Student operations	<code>issue_a_book()</code> , <code>return_a_book()</code>

Book record structure

```
{ "Book name": "python basics",    # stored in lower case
  "Id": "B1",
  "author name": "Guido van Rossum",
  "category": "Programming",
  "available": 2,                  # copies on the shelf
  "reg_no": [],                   # students currently holding a copy
  "waiting_list": [] }            # queue of students waiting
```

Because lists and dictionaries are mutable, functions modify `list_of_books` in place. Nothing needs to be returned to the main loop, and every part of the program sees the same up-to-date state.

7. Design Diagrams

7.1 Use Case Diagram

Two actors interact with the system. The Librarian adds books and checks status, and both actions include the authorisation step. The Student issues and returns books; issuing may extend into joining the waiting list, and returning may extend into fine calculation.

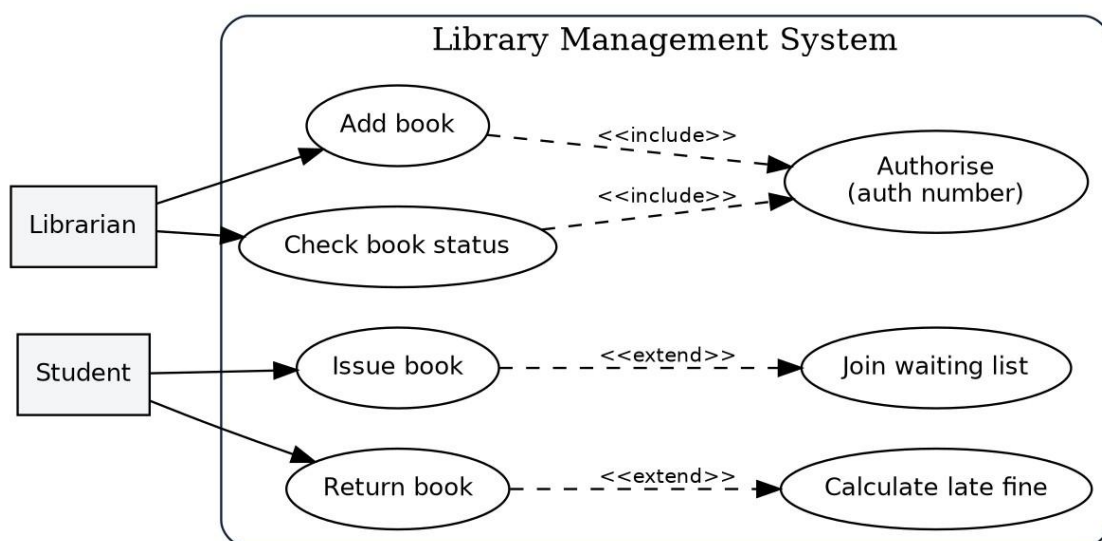


Figure 1: Use case diagram

7.2 Workflow Diagram

The workflow shows the main loop. Every branch returns to the menu until the user selects option 5.

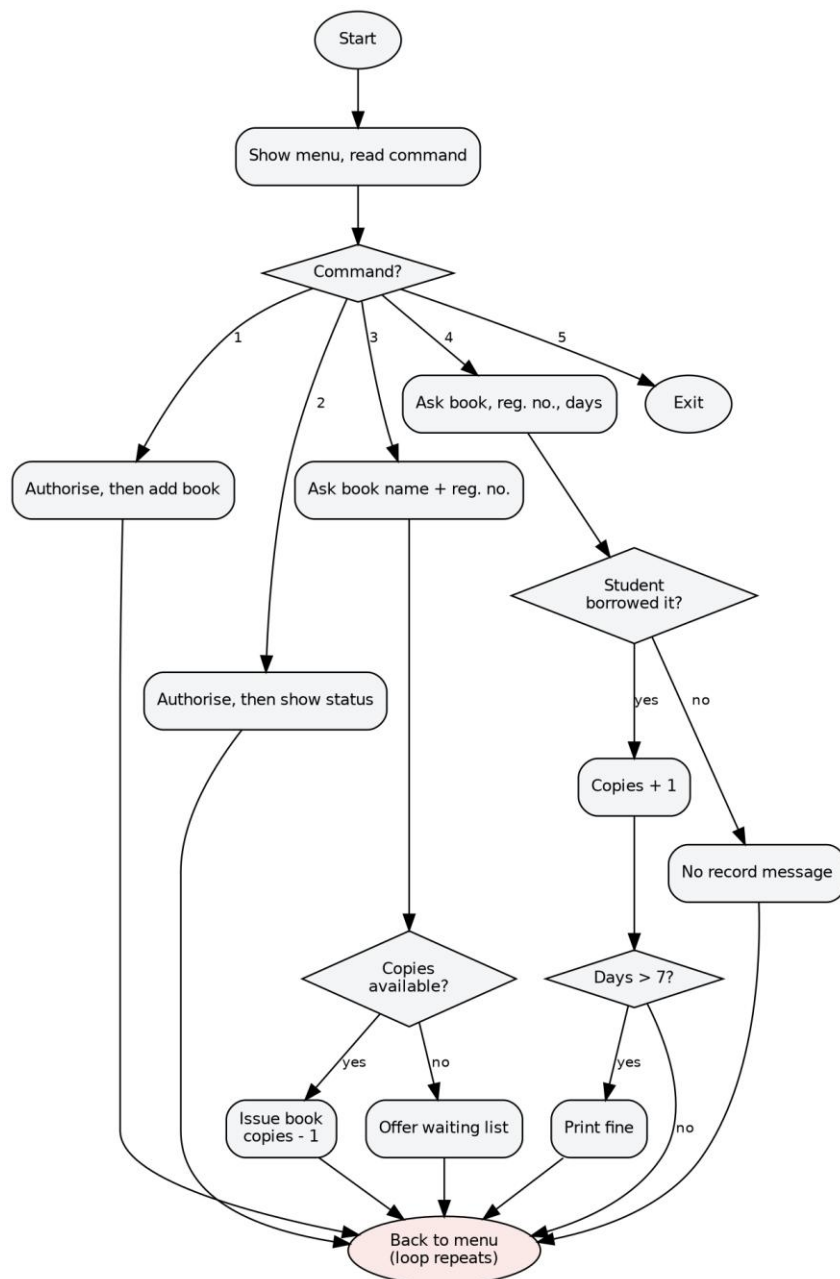


Figure 2: Workflow diagram of the menu loop

7.3 Sequence Diagrams

Three key interactions are shown: authorisation, issuing a book and returning a book.

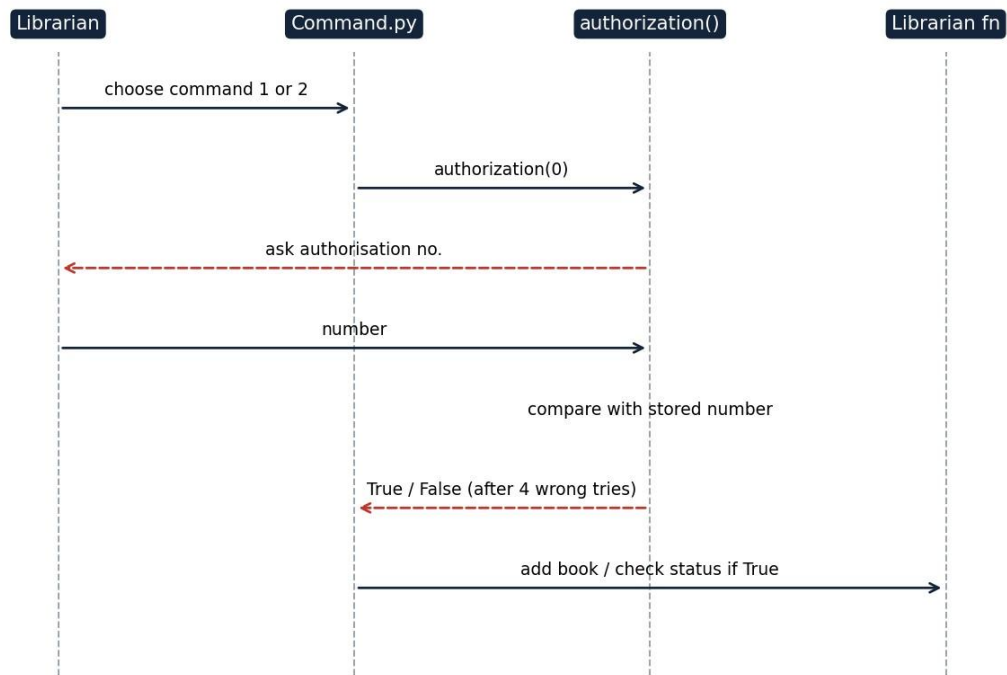


Figure 3: Sequence diagram - librarian authorisation

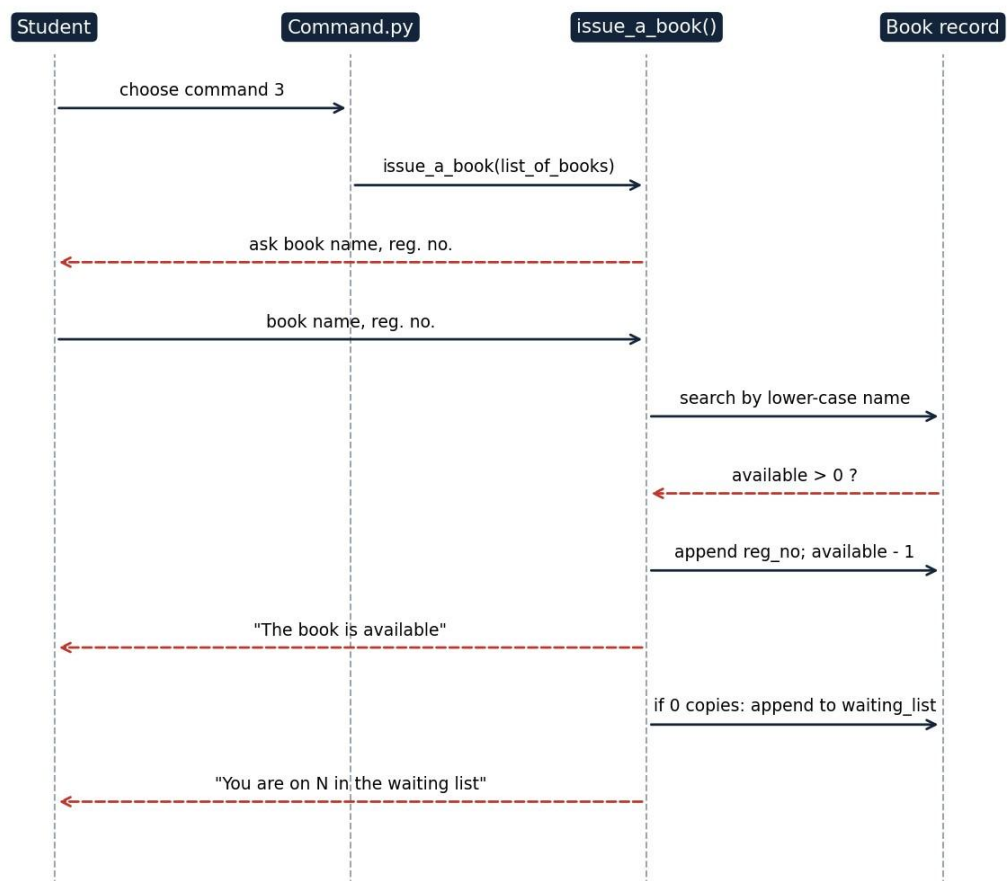


Figure 4: Sequence diagram - issuing a book (with waiting list)

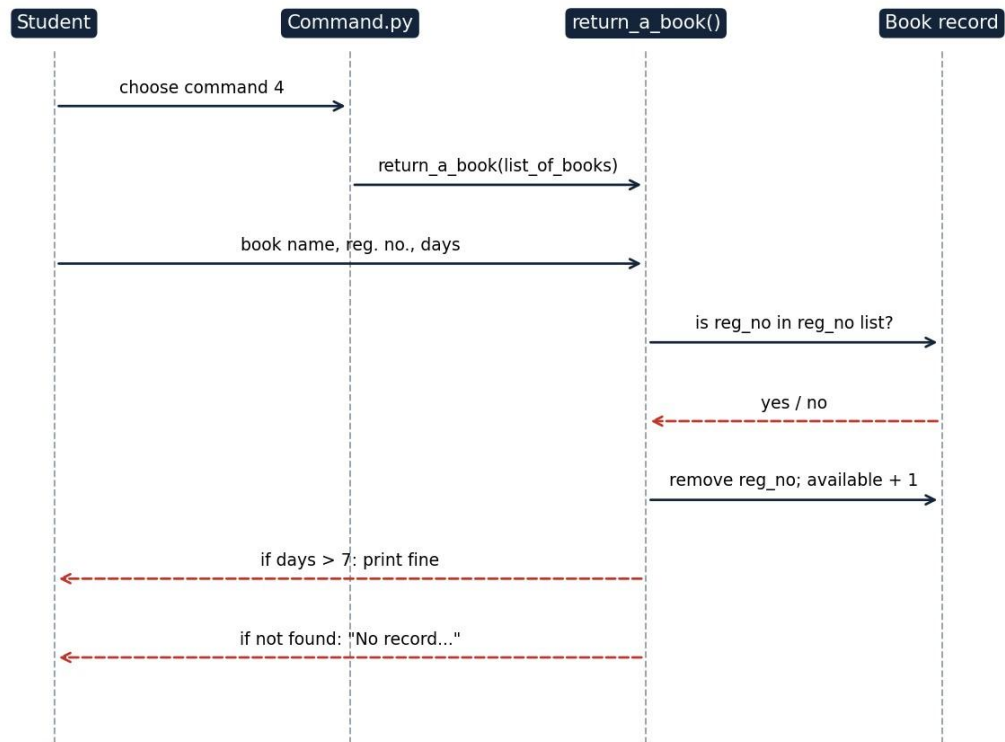


Figure 5: Sequence diagram - returning a book

7.4 Class / Component Diagram

The program is procedural rather than object-oriented, so it is modelled as components. The entry point imports the two functional modules, and both operate on the shared book record.

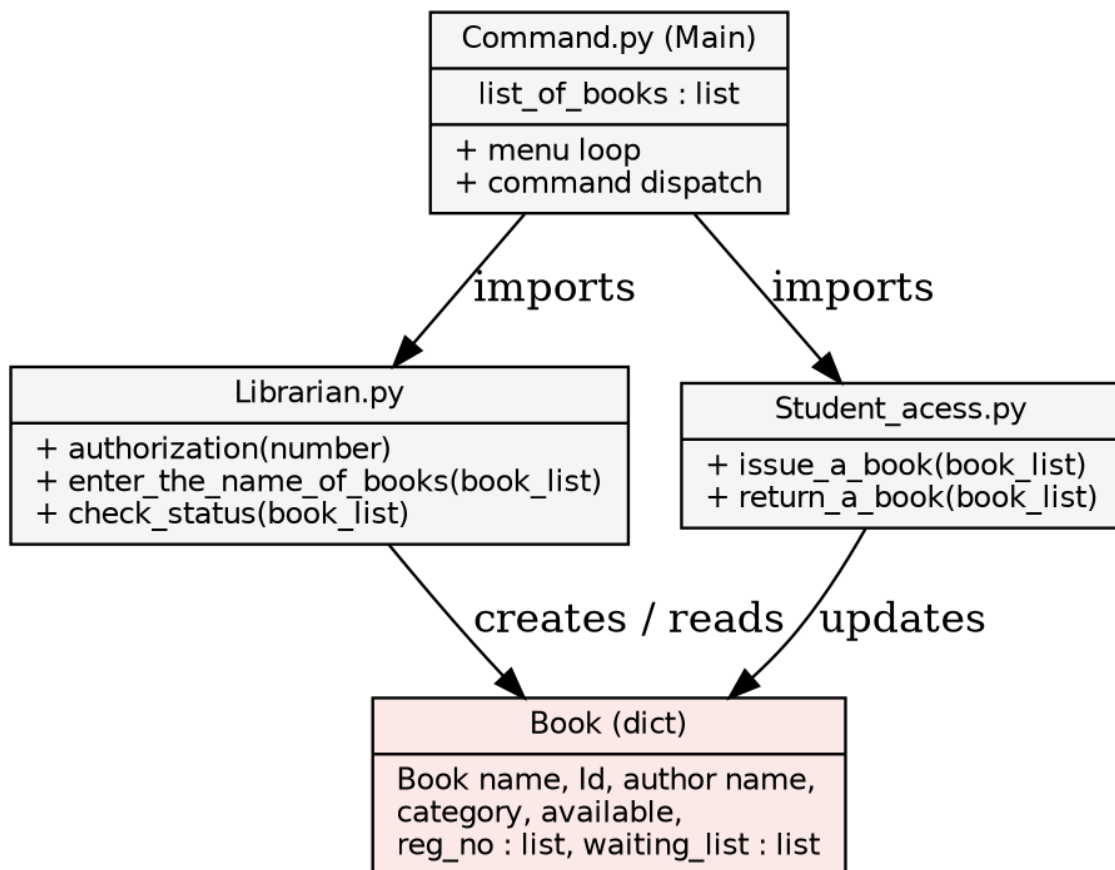


Figure 6: Component diagram

7.5 ER Diagram

The application does not use a database; all data lives in memory. The diagram below is the **logical data model** implied by the code, and shows how the book dictionary would map to relational tables if persistent storage were added. In the current code, `BORROW` corresponds to the `reg_no` list and `WAITING_LIST` to the `waiting_list` list inside each book.

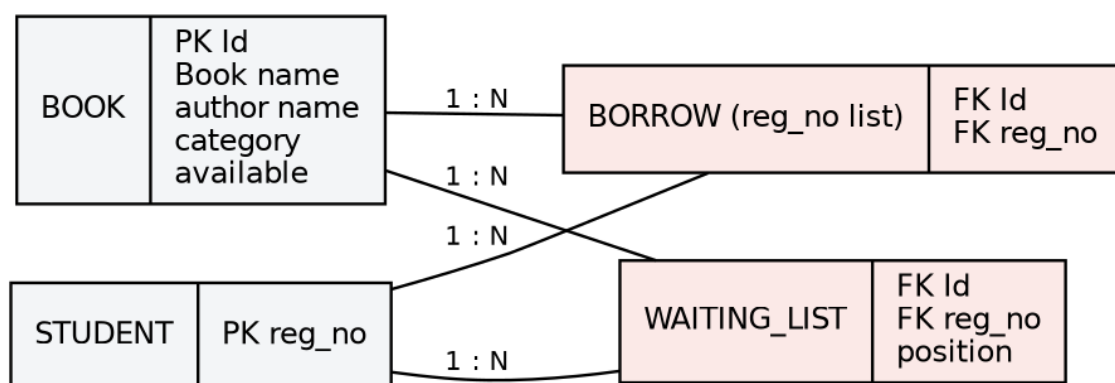


Figure 7: Logical ER diagram (conceptual, not physically stored)

8. Design Decisions & Rationale

Decision	Rationale
----------	-----------

Console menu instead of a GUI	Keeps the focus on program logic and makes the tool runnable anywhere with no dependencies.
Decision	Rationale
Separate modules for librarian and student	Mirrors the two user roles, keeps each file short, and lets each role be changed without touching the other.
Books as dictionaries in a list	The simplest structure that holds mixed data (text, count, lists) per book and is easy to print and iterate; suitable for a small collection.
Names and registration numbers lower-cased	Removes case mismatches such as "Python Basics" vs "python basics" without extra user effort.
Per-book borrower and waiting lists	Keeping them inside the book record makes it easy to see, for one book, who holds it and who is waiting, and makes waiting position simply the list length.
Authorisation number with limited attempts	A lightweight way to separate staff from student functions and to stop unlimited guessing.
Validate borrower on return	Prevents the copy count from growing when someone who never borrowed a book "returns" it.
Fine as base amount plus weekly increment	A simple, predictable rule: 250 once the 7-day limit is passed, plus 10 for each additional full week.

9. Implementation Details

The project is written in Python 3 and uses only built-in features. The key routines are described below.

9.1 Main loop (Command.py)

The entry point creates `list_of_books`, prints the authorisation number, and loops forever, reading a command with `int(input())`. An `if/elif` chain sends command 1 and 2 through `authorization()` first, calls the student functions for 3 and 4, breaks the loop for 5, and prints "Invalid command" otherwise.

9.2 Authorisation (Librarian.py) `authorization()` compares each entry with the stored number 8624357. The loop runs while `attempt <= 3`, so four entries are allowed. After each wrong entry the program reports how many attempts remain; after the fourth it prints "Trial limit reached" and "Access denied" and returns False.

9.3 Adding a book and checking status (Librarian.py)

`enter_the_name_of_books()` reads the five book details and appends a dictionary with an empty borrower list and an empty waiting list. `check_status()` iterates through every book and prints each key and value.

9.4 Issuing a book (Student_access.py)

The function lower-cases the book name and registration number and scans the list. If the name matches and `available > 0`, the registration number is appended to the borrower list and the

count is decreased. If the name matches but no copies remain, the student is asked whether to join the waiting list; on "yes" they are appended and told their position (the list length). If no book matches, the student is told the book is not in the library.

9.5 Returning a book and fine calculation (Student_aces.py)

The function reads the book name, registration number and the number of days kept. If the student is not in the borrower list, it prints "No record of this student borrowing this book." Otherwise the registration number is removed and the count increased. When `days > 7` the fine is:

```
fine = 250 + ((days - 8) // 7) * 10
```

Days kept	Fine
7 or fewer	0 (no message)
8 to 14	250
15 to 21	260
22 to 28	270

10. Screenshots / Results

The screenshots below were produced by running the program with the inputs shown; they are the real output of the submitted code. To save space, the repeated five-line main menu is shown as "[Main menu: options 1-5]".

```

8624357 is the authorization number
[ Main menu: options 1-5 ]
Enter the command:1
Enter the authorisation no.:8624357
Enter book name:Python Basics
Enter book ID:B1
Enter the author name:Guido van Rossum
Enter category of the book:Programming
Enter the number of books:2
[ Main menu: options 1-5 ]
Enter the command:2
Enter the authorisation no.:8624357
This is the status of the books
Book name:python basics
Id:B1
author name:Guido van Rossum
category:Programming
available:2
reg_no:[]
waiting_list:[]
[ Main menu: options 1-5 ]
Enter the command:5
Ok! terminating

```

Figure 8: Adding a book and viewing status (commands 1 and 2)

```

8624357 is the authorization number
[ Main menu: options 1-5 ]
Enter the command:1
Enter the authorisation no.:1
Access denied
Try again
You have 3 left
Enter the authorisation no.:2
Access denied
Try again
You have 2 left
Enter the authorisation no.:3
Access denied
Try again
You have 1 left
Enter the authorisation no.:4
Trial limit reached
Access denied
[ Main menu: options 1-5 ]
Enter the command:5
Ok! terminating

```

Figure 9: Four wrong authorisation attempts lead to "Access denied"

```
8624357 is the authorization number
[ Main menu: options 1-5 ]
Enter the command:1
Enter the authorisation no.:8624357
Enter book name:Python Basics
Enter book ID:B1
Enter the author name:Guido van Rossum
Enter category of the book:Programming
Enter the number of books:2
[ Main menu: options 1-5 ]
Enter the command:3
Enter the book name:Python Basics
Enter your registration number:21BCS001
The book is available
Here remember to return the book
[ Main menu: options 1-5 ]
Enter the command:3
Enter the book name:python basics
Enter your registration number:21bcs002
The book is available
Here remember to return the book
[ Main menu: options 1-5 ]
Enter the command:3
Enter the book name:Python Basics
Enter your registration number:21bcs003
The book is not available
But do you want to be in waitlist ?yes
Okay you are on 1 in the waiting list.
We will notify you once the book is ready to be issued
[ Main menu: options 1-5 ]
Enter the command:3
Enter the book name:Data Science
Enter your registration number:21bcs004
There is no such book in our library
[ Main menu: options 1-5 ]
Enter the command:5
Ok! terminating
```

Figure 10: Issuing books, joining the waiting list, and requesting an unknown book

```
8624357 is the authorization number
[ Main menu: options 1-5 ]
Enter the command:1
Enter the authorisation no.:8624357
Enter book name:Python Basics
Enter book ID:B1
Enter the author name:Guido van Rossum
Enter category of the book:Programming
Enter the number of books:2
[ Main menu: options 1-5 ]
Enter the command:3
Enter the book name:Python Basics
Enter your registration number:21bcs001
The book is available
Here remember to return the book
[ Main menu: options 1-5 ]
Enter the command:4
Enter the book namePython Basics
Enter your registration number21bcs001
Enter the number of days the book was borrowed for:8
The amount of fine is 250
Return the book on time next time!!
[ Main menu: options 1-5 ]
Enter the command:3
Enter the book name:Python Basics
Enter your registration number:21bcs002
The book is available
Here remember to return the book
[ Main menu: options 1-5 ]
Enter the command:4
Enter the book namePython Basics
Enter your registration number21bcs002
Enter the number of days the book was borrowed for:15
The amount of fine is 260
Return the book on time next time!!
[ Main menu: options 1-5 ]
Enter the command:4
Enter the book namePython Basics
Enter your registration number99xyz
Enter the number of days the book was borrowed for:3
No record of this student borrowing this book.
[ Main menu: options 1-5 ]
Enter the command:9
Invalid command
[ Main menu: options 1-5 ]
Enter the command:5
Ok! terminating
```

Figure 11: Returning books - fine of 250 (8 days), fine of 260 (15 days), wrong student, invalid command

11. Testing Approach

Testing was manual and scenario-based: each feature was exercised through the console with normal, boundary and invalid inputs, and the observed output was compared with the expected result. The scenarios below were run against the submitted code; the "Observed" column reflects the actual output.

#	Scenario	Input	Expected	Observed
---	----------	-------	----------	----------

1	Correct authorisation	Command 1, 8624357	Prompts for book details	Pass
2	Wrong authorisation	Command 1, four wrong numbers	Attempts counted down, then "Access denied"	Pass (3, 2, 1 left, then denied)
3	Add a book	Python Basics, B1, 2 copies	Book stored	Pass
4	Check status	Command 2	All details listed	Pass
5	Issue available book	Python Basics, 21bcs001	"The book is available"; copies 2 to 1	Pass
6	Issue unavailable book	Third request after 2 copies issued; answer yes	Not available; waiting position 1	Pass
7	Book not in library	Data Science	"There is no such book in our library"	Pass
8	Case-insensitive match	"python basics", "21BCS001"	Matches stored record	Pass
9	Return on time	7 days	Copy returned, no fine	Pass (no fine; no confirmation message is printed)
10	Return late	8 days	Fine 250	Pass
11	Return late (longer)	15 days	Fine 260	Pass
12	Return by wrong student	Reg. no. 99xyz	"No record of this student borrowing this book."	Pass
13	Invalid menu input	Command 9	"Invalid command"	Pass

Scenario 9 was checked by inspection of the run output only for the fine; boundary behaviour at exactly 7 days follows from the `days > 7` condition.

12. Challenges Faced

- **Limiting authorisation attempts.** Designing the retry loop so the counter, the "attempts left" message and the final denial stayed consistent required careful handling of the attempt variable.
- **Keeping matching reliable.** Users type names and registration numbers in different cases, so all comparisons had to be done on lower-cased values, and books had to be stored in lower case.
- **Modelling the waiting list.** Storing the waiting list inside each book made the position trivial to compute, but required deciding what should happen to it on return (currently it is only recorded, see Limitations).

- **Validating returns.** The return function had to distinguish "book not found", "student never borrowed it" and "valid return" without wrongly increasing the copy count.
- **Fine rule.** Turning the rule into a formula that gives 250 at 8 days and adds 10 per extra full week meant working out the integer division carefully and testing the boundary days.
- **Sharing state across modules.** Passing one list by reference to every function avoided global variables while keeping all modules in sync.

13. Limitations & Future Enhancements

The following are honest limitations of the current implementation, with the enhancement that would address each.

Limitation	Proposed enhancement
Data is stored in memory only and is lost when the program exits.	Persist books to a file (JSON/CSV) or a SQLite database, following the ER model in section 7.5.
The authorisation number is hard-coded and is printed at start-up.	Read a secret from an environment variable or config file, hash it, and stop displaying it.
Students on the waiting list are not automatically offered a book when one is returned.	On return, pop the first waiting student and issue the copy (or notify them).
Non-numeric input for the menu, copies or days raises an error and stops the program.	Wrap input in try/except and re-prompt.
The number of days is typed in by the student instead of computed from issue and return dates.	Store the issue date and compute days with the datetime module.
A successful on-time return prints no confirmation, and returning a title that does not exist prints nothing.	Add confirmation and "book not found" messages.
The same student can issue the same book more than once; duplicate book titles are not checked.	Reject duplicate borrowing and merge or reject duplicate titles.
No automated tests.	Add unit tests (unittest/pytest) for the functions, using mocked input.

14. Conclusion

The Library Management System meets its stated objectives: a librarian can add and inspect books behind an authorisation check, students can issue and return books with the copy count kept correct, unavailable books can be queued, and late fines are computed by a clear rule. Manual testing across normal, boundary and invalid inputs showed the program behaving as designed. The modular structure keeps the code easy to read and extend, and the limitations listed above give a clear path from this console prototype towards a persistent, more robust system.

Repository: <https://github.com/aditirastogi-code/Library-management-system>